Enterprise AI Chatbot Implementation Guide

Modular Architecture: FastAPI + LangGraph (Phases 1-5) + Persistent ChromaDB + Groq

Target Stack: FastAPI, LangGraph, ChromaDB, Groq

Architecture: ReAct + Persistent SQLite + HITL

Vector Storage: Disk-backed ChromaDB (ONNX Embeddings)

Frontend: Web Chat UI rendered via FastAPI

Project Architecture & Phases Mapping

Phase	Core Concept	File Location
Phase 1	StateGraph, Nodes, & TypedDict Schema	app/state.py, app/graph.py
Phase 2	Reducers (add_messages, operator.add)	app/state.py
Phase 3	Tools & Cyclic ReAct loop	app/tools.py, app/graph.py
Phase 4	Persistent SQLite thread memory (SqliteSaver)	app/graph.py
Phase 5	Human-in-the-Loop approval gate (interrupt_before)	app/graph.py, app/main.py
RAG / Cache	Unified ChromaDB Q&A; semantic memory	app/database.py
Delivery	FastAPI endpoints + Web UI	app/main.py, app/ui.html

Part 0: Project Setup & Dependencies

Create the directory structure and install the requirements:

- Directory: my_chatbot_project/app/
- Dependencies: requirements.txt

```
langgraph>=0.2.0
langgraph-checkpoint-sqlite>=3.1.0
langchain-core>=0.3.0
langchain-groq>=0.2.0
langchain-chroma>=0.1.0
chromadb>=0.5.0
fastapi>=0.110.0
uvicorn>=0.29.0
python-dotenv>=1.0.0
pydantic>=2.0.0
```

Part 1: State & Reducers (app/state.py)

Defines the state structure that flows between graph nodes. Reducers dictate how updates merge:

- **add_messages:** Appends conversation messages and enables update-by-id.
- **operator.add:** Appends entries to the audit trail log.
- **sensitive_action_pending:** Standard boolean flag that overwrites on update.

```
from typing import Annotated, TypedDict, Optional, Any
import operator
from langgraph.graph.message import add_messages

class ChatbotState(TypedDict):
    messages: Annotated[list, add_messages]
    audit_log: Annotated[list[str], operator.add]
    sensitive_action_pending: bool
    pending_action_details: Optional[dict[str, Any]]
```

Part 2: Unified Persistent ChromaDB (app/database.py)

Stores official policies and newly answered user questions as (Question, Answer) pairs on disk using cosine vector space:

```
import os, time, uuid
from typing import Optional, Any
import chromadb
from chromadb.utils import embedding_functions

DATA_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "data")
CHROMA_PERSIST_DIR = os.path.join(DATA_DIR, "chroma_db")
os.makedirs(CHROMA_PERSIST_DIR, exist_ok=True)

_client = chromadb.PersistentClient(path=CHROMA_PERSIST_DIR)
_emb_fn = embedding_functions.DefaultEmbeddingFunction()

qa_collection = _client.get_or_create_collection(
    name="qa_knowledge_base",
    embedding_function=_emb_fn,
    metadata={"hnsw:space": "cosine"}
)

if qa_collection.count() == 0:
    seed_qa_pairs = [
        {"question": "What is your return policy?", "answer": "Returns within 30 days get 100% refund. Afte
r 30 days: store credit."},
        {"question": "What are your support hours?", "answer": "Support is active 9:00 AM to 6:00 PM EST, M
onday-Friday."},
        {"question": "What are transfer limits?", "answer": "Credits over $100 require Human Manager approv
al."}
    ]
    qa_collection.add(
        documents=[item["question"] for item in seed_qa_pairs],
        ids=[f"seed_{i+1}" for i in range(len(seed_qa_pairs))],
        metadatas=[{"answer": item["answer"]} for item in seed_qa_pairs]
    )

def store_qa_pair(question: str, answer: str, thread_id: str) -> str:
    qa_id = f"qa_{uuid.uuid4().hex[:10]}"
    qa_collection.add(
        documents=[question],
        ids=[qa_id],
```

```
        metadatas=[{"answer": answer, "thread_id": thread_id, "timestamp": time.strftime("%Y-%m-%d %H:%M:%S
")}]
    )
    return qa_id

def find_similar_past_qa(query: str, similarity_threshold: float = 0.55) -> Optional[dict[str, Any]]:
    if qa_collection.count() == 0: return None
    res = qa_collection.query(query_texts=[query], n_results=1, include=["documents", "metadatas", "distanc
es"])
    docs, metas, dists = res.get("documents", [[]])[0], res.get("metadatas", [[]])[0], res.get("distances",
 [[]])[0]
    if docs and dists and dists[0] <= similarity_threshold:
        return {"matched_question": docs[0], "matched_answer": metas[0].get("answer", ""), "distance": dist
s[0]}
    return None
```

Part 3: Action Tools (app/tools.py)

Operational tools the agent executes during reasoning:

```
from langchain_core.tools import tool

@tool
def execute_account_credit(user_id: str, amount: float, reason: str) -> str:
    """Issues an account credit or refund to a user. Credits > $100 require approval."""
    if amount > 100.0:
        return f"ALERT: Amount ${amount:.2f} exceeds $100. Placed on HOLD for supervisor approval."
    return f"SUCCESS: Credit of ${amount:.2f} applied to {user_id}."

ALL_TOOLS = [execute_account_credit]
```

Part 4: LangGraph Agent Engine (app/graph.py)

Compiles the state machine with cyclic loop, SQLite checkpointer, and HITL gate:

```
import os, sqlite3
from dotenv import load_dotenv
from langchain_groq import ChatGroq
from langchain_core.messages import SystemMessage, AIMessage
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode
from langgraph.checkpoint.sqlite import SqliteSaver
from app.state import ChatbotState
from app.tools import ALL_TOOLS

load_dotenv()
llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0)
llm_with_tools = llm.bind_tools(ALL_TOOLS)

SYSTEM_PROMPT = SystemMessage(content="You are a customer assistant. For credits/refunds, call execute_acco
unt_credit.")

def agent_node(state: ChatbotState) -> dict:
    resp = llm_with_tools.invoke([SYSTEM_PROMPT] + state["messages"])
    sensitive = False
    details = None
    if hasattr(resp, "tool_calls") and resp.tool_calls:
        for tc in resp.tool_calls:
            if tc["name"] == "execute_account_credit" and tc.get("args", {}).get("amount", 0) > 100:
                sensitive = True
                details = {"tool": "execute_account_credit", "args": tc["args"]}
    return {"messages": [resp], "audit_log": [f"agent: turn done (tool_calls={bool(resp.tool_calls)})"],
            "sensitive_action_pending": sensitive, "pending_action_details": details}

def sensitive_action_gate_node(state: ChatbotState) -> dict:
    args = (state.get("pending_action_details") or {}).get("args", {})
    return {"messages": [AIMessage(content=f"Human Supervisor APPROVED credit of ${args.get('amount', 0):.2
f}.")],
            "audit_log": ["supervisor: transaction settled"], "sensitive_action_pending": False, "pending_a
ction_details": None}

def custom_router(state: ChatbotState) -> str:
    if state.get("sensitive_action_pending"): return "sensitive_action_gate"
```

```
    last = state["messages"][-1]
    return "tools" if (hasattr(last, "tool_calls") and last.tool_calls) else END

builder = StateGraph(ChatbotState)
builder.add_node("agent", agent_node)
builder.add_node("tools", ToolNode(tools=ALL_TOOLS))
builder.add_node("sensitive_action_gate", sensitive_action_gate_node)

builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", custom_router, {"tools": "tools", "sensitive_action_gate": "sensitiv
e_action_gate", END: END})
builder.add_edge("tools", "agent")
builder.add_edge("sensitive_action_gate", END)

DATA_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "data")
db_conn = sqlite3.connect(os.path.join(DATA_DIR, "chat_checkpoints.db"), check_same_thread=False)
chatbot_graph = builder.compile(checkpointer=SqliteSaver(db_conn), interrupt_before=["sensitive_action_gate
"])
```

Part 5 & 6: FastAPI Application (app/main.py)

Orchestrates semantic cache checks, graph execution, HITL approval, and UI rendering:

```
import os
from fastapi import FastAPI
from fastapi.responses import HTMLResponse
from pydantic import BaseModel
from langchain_core.messages import HumanMessage, AIMessage
from langchain_groq import ChatGroq
from app.graph import chatbot_graph
from app.database import store_qa_pair, find_similar_past_qa

app = FastAPI(title="Enterprise Agent")
adapter_llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0)
UI_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui.html")

class ChatRequest(BaseModel):
    message: str
    thread_id: str = "default_thread"

class ApprovalRequest(BaseModel):
    thread_id: str
    decision: str
    modified_amount: float = None

@app.get("/", response_class=HTMLResponse)
def get_ui():
    with open(UI_PATH, "r", encoding="utf-8") as f: return f.read()

@app.post("/chat")
def chat(payload: ChatRequest):
    config = {"configurable": {"thread_id": payload.thread_id}}
    is_tx = any(w in payload.message.lower() for w in ["credit", "refund", "dollar", "$"])

    # Check Semantic Q&A Cache first
    match = None if is_tx else find_similar_past_qa(payload.message)
    if match:
        prompt = f"User asked: '{payload.message}'. Base answer: '{match['matched_answer']}'. Adapt natural
ly."
```

```
        adapted = adapter_llm.invoke(prompt).content
        chatbot_graph.update_state(config, {"messages": [HumanMessage(content=payload.message), AIMessage(c
ontent=adapted)],
                                           "audit_log": ["cache: Adapted past Q&A"]})
        curr = chatbot_graph.get_state(config)
        return {"reply": adapted, "source": "UNIFIED_QA_ADAPTED", "status": "COMPLETED", "checkpoint_id": c
urr.config["configurable"]["checkpoint_id"]}

    # Run LangGraph Agent
    chatbot_graph.invoke({"messages": [HumanMessage(content=payload.message)], "audit_log": [f"user: {paylo
ad.message[:30]}"]}, config=config)
    curr = chatbot_graph.get_state(config)
    is_paused = "sensitive_action_gate" in curr.next
    last_msg = curr.values["messages"][-1]

    if not is_paused and not is_tx:
        store_qa_pair(payload.message, last_msg.content, payload.thread_id)

    return {"reply": last_msg.content if not is_paused else "On HOLD awaiting approval.",
            "source": "AGENT_GRAPH_COMPUTED", "status": "PAUSED_AWAITING_APPROVAL" if is_paused else "COMPL
ETED",
            "pending_action": curr.values.get("pending_action_details")}

@app.post("/approve")
def approve(payload: ApprovalRequest):
    config = {"configurable": {"thread_id": payload.thread_id}}
    curr = chatbot_graph.get_state(config)
    if payload.modified_amount is not None:
        d = curr.values.get("pending_action_details") or {}
        if "args" in d: d["args"]["amount"] = payload.modified_amount
        chatbot_graph.update_state(config, {"pending_action_details": d, "audit_log": [f"supervisor: change
d to ${payload.modified_amount}"]})
    final_s = chatbot_graph.invoke(None, config=config)
    return {"status": "APPROVED", "final_reply": final_s["messages"][-1].content}
```

Part 7: Running & Verifying

1. Start the server with uvicorn:

```
python -m uvicorn app.main:app --reload --port 8000
```

2. Open your browser at **http://127.0.0.1:8000/** to use the web interface.

3. Test the interactive flow:

- **Q&A; Matching:** Ask *'What is your return policy?'* → adapted immediately.
- **HITL Safety Gate:** Ask *'Please credit $250 to user_101'* → pauses with approval bar.
- **Thread Isolation:** Switch threads in top bar to test separate memories.